

CHECK POINT 01

Ferramentas open source de segurança

CLOUD SECURITY | Professor Fabio Pires

Leonardo Garroti • Aquiles Fonseca • Leandro de Souza

Objetivo do trabalho

- Entender quatro tipos de teste de segurança
- Estudar as ferramentas indicadas ao grupo 3
- Executar njsscan e npm audit no Docker
- Comparar uma versão insegura com outra corrigida

Quatro formas de olhar a segurança

- SAST: verifica o código
- SCA: verifica as bibliotecas
- IaC: verifica arquivos de infraestrutura
- DAST: testa a aplicação funcionando

Onde cada teste entra

- Ao escrever o código: SAST
- Ao instalar dependências: SCA
- Antes do deploy: análise de IaC
- No ambiente de teste: DAST
- Testar cedo ajuda a corrigir antes da entrega

njsscan

- Scanner SAST para Node.js
- Analisa o código sem iniciar a aplicação
- Gera relatórios JSON, SARIF e HTML
- Versão usada no laboratório: 1.0.0
- Licença indicada pelo projeto: LGPL-3.0

O que ele procura

Padrões perigosos, como entrada do usuário enviada para HTML, comandos do sistema ou eval.

Resultado real do njsscan

Três alertas na versão insegura

```
njsscan: v1.0.0 | Ajin Abraham | opensecurity.in
RULE ID      express_xss
OWASP-WEB    A1: Injection
CWE          CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
DESCRIPTION  Untrusted User Input in Response will result in Reflected Cross Site Scripting Vulnerability.
SEVERITY     ERROR

File         app.js
Match Position 3 - 43
FILES        Line Number(s) 8: 9
Match String  const nome = req.query.nome;
               res.send('<h1>Olá, ' + nome + '</h1>');

RULE ID      generic_os_command_exec
OWASP-WEB    A1: Injection
CWE          CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
DESCRIPTION  User controlled data in 'child_process.exec()' can result in Remote OS Command Execution.
SEVERITY     ERROR

File         app.js
Match Position 3 - 5
FILES        Line Number(s) 12: 14
Match String  exec(req.query.cmd, function (erro, saida) {
               res.type('text').send(erro ? 'Erro' : saida);
               });

RULE ID      eval_nodejs
OWASP-WEB    A1: Injection
CWE          CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')
DESCRIPTION  User controlled data in eval() or similar functions may result in Server Side Injection or Remote Code Injection
SEVERITY     ERROR

File         app.js
Match Position 21 - 41
FILES        Line Number(s) 17
Match String  const resultado = eval(req.query.expr);
```

Os três achados SAST

- CWE-79: entrada do usuário enviada para HTML
- CWE-78: entrada usada em comando do sistema
- CWE-95: texto enviado para eval
- Os três casos foram confirmados no código
- Classificação: verdadeiros positivos

Como o código foi corrigido

- Resposta HTML trocada por texto simples
- Rota de comando passou a responder uma mensagem fixa
- eval foi retirado
- A soma agora aceita apenas números válidos
- Nenhuma regra foi escondida ou ignorada

npm audit

- Ferramenta SCA do npm
- Compara dependências com avisos conhecidos
- Usa o package-lock.json
- É simples de incluir no pipeline
- Zero alertas conhecidos não significa risco zero

Resultado real do npm audit

lodash 4.17.15: um pacote com severidade HIGH

```
LAB GRUPO 3 - NPM AUDIT x + v
COMANDO: docker compose run --rm lab (npm audit na versão insegura)

# npm audit report

lodash <=4.17.23
Severity: high
Command Injection in lodash - https://github.com/advisories/GHSA-35jh-r3h4-6jhm
Prototype Pollution in lodash - https://github.com/advisories/GHSA-p6mc-m468-83gw
Regular Expression Denial of Service (ReDoS) in lodash - https://github.com/advisories/GHSA-29mw-wpgm-hmr9
lodash vulnerable to Code Injection via `_.template` imports key names - https://github.com/advisories/GHSA-r5fr-rjxr-66jc
lodash vulnerable to Prototype Pollution via array path bypass in `_.unset` and `_.omit` - https://github.com/advisories/GHSA-f23m-r3pf-42rh
Lodash has Prototype Pollution Vulnerability in `_.unset` and `_.omit` functions - https://github.com/advisories/GHSA-xxjr-mmjv-4gpg
fix available via `npm audit fix --force`
Will install lodash@4.18.1, which is outside the stated dependency range

1 high severity vulnerability

To address all issues, run:
  npm audit fix --force

CODIGO_DE_SAIDA=1
|
```

Leitura correta do resultado

- O lockfile confirmou a versão vulnerável
- O alerta de inventário é verdadeiro
- Nem todo aviso é explorável pelas rotas do exemplo
- A correção foi atualizar o lodash para 4.18.1
- O projeto precisa ser testado depois da atualização

Datree

- Analisava arquivos Kubernetes antes do deploy
- Aplicava políticas sobre privilégios e recursos
- Projeto arquivado no GitHub
- Ainda ajuda no estudo, mas deixou de ser uma boa escolha para projetos novos
- Foi pesquisado, porém não foi executado no laboratório

Exemplo de infraestrutura como código

- Arquivo inseguro: contêiner privilegiado e sem limites
- Arquivo corrigido: sem privilégio elevado
- Sistema de arquivos somente para leitura
- Limites de CPU e memória definidos
- A verificação acontece antes de chegar ao cluster

StackHawk e HawkScan

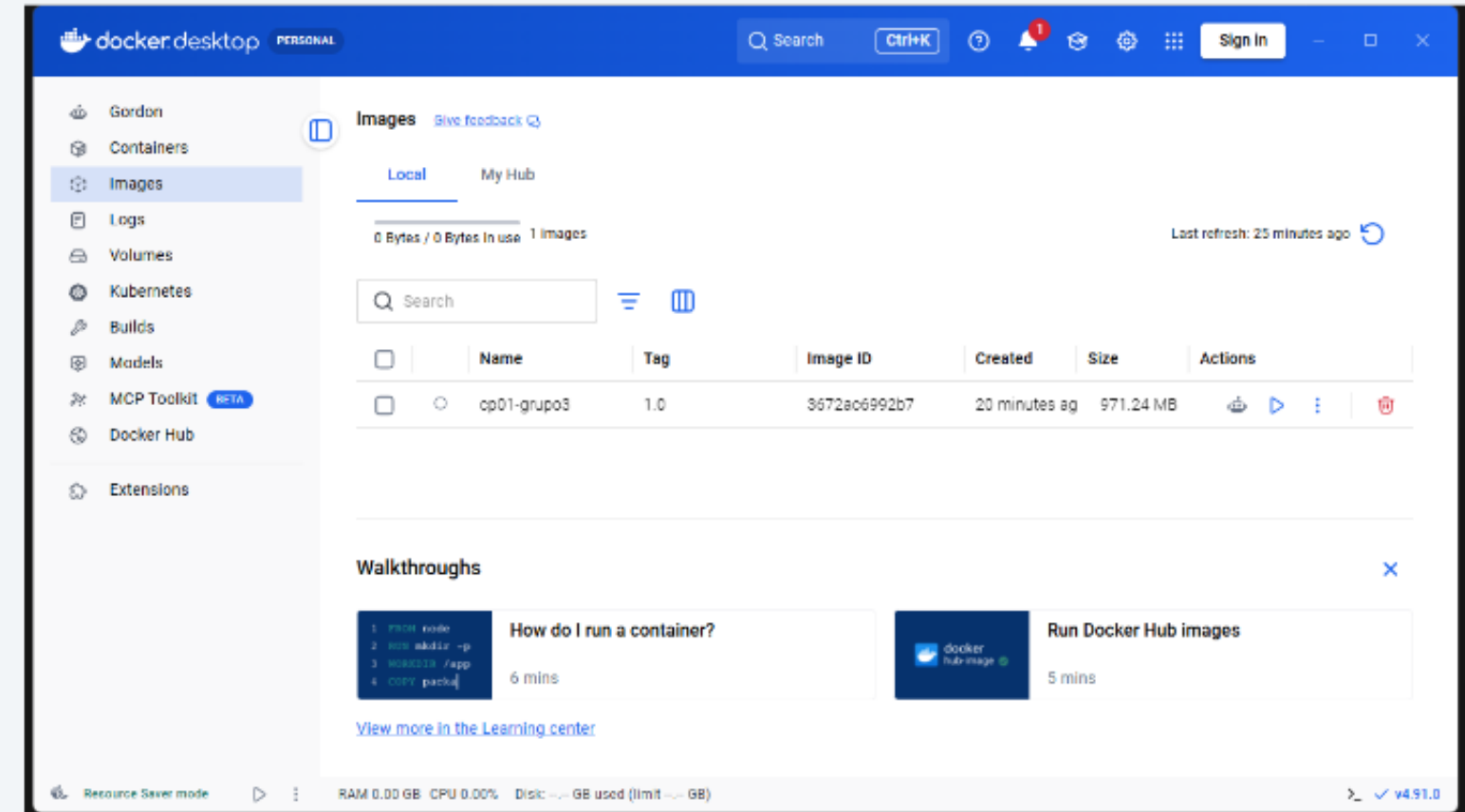
- Ferramenta DAST para testar a aplicação em execução
- O scanner tem base ligada ao OWASP ZAP
- Precisa conhecer o endereço autorizado do teste
- Rotas protegidas exigem configuração de login
- A plataforma possui partes comerciais

Como funciona um teste DAST

- Inicia a aplicação de teste
- Define o endereço permitido
- Envia requisições para as rotas
- Revisa os alertas encontrados
- Corrige e executa novamente

Ambiente do laboratório

- Docker Desktop com WSL 2
- Imagem: cp01-grupo3:1.0
- Node.js, Python, njsscan, Semgrep e npm
- Duas versões da mesma aplicação própria



Versões e imagem do Docker

```
LAB GRUPO 3 - DOCKER x + v  
COMANDO: docker compose build --pull lab  
  
Docker version 29.8.0, build 88096ef  
Docker Compose version v5.5.1  
WSL: Ubuntu, versão 2  
  
Imagem: cp01-grupo3:1.0  
ID: sha256:3672ac6992b77b006867348d9b05d2015cb33269a723ac86014ce11688ef4f25  
Criada em: 2026-09-16T18:55:28.743786323Z  
Tamanho: 971245040 bytes  
  
RESULTADO: imagem criada com sucesso.  
|
```

Política do gate

- SAST ERROR é tratado como HIGH
- SCA HIGH ou CRITICAL bloqueia
- Sem alertas nesses níveis, a versão passa
- A política é simples e foi criada para o laboratório
- O código de saída permite bloquear o pipeline

Versão insegura bloqueada

3 alertas SAST + 1 pacote HIGH = BLOQUEADO

```
LAB GRUPO 3 - TESTE BLOQU  X  +  v
COMANDO: docker compose run --rm lab inseguro

SAIDA REAL DO CONTAINER:
{
  "alvo": "inseguro",
  "status": "BLOQUEADO",
  "exit_code": 1,
  "politica": "SAST ERROR = HIGH por decisão do grupo; SCA HIGH/CRITICAL bloqueiam",
  "contagens": {
    "sast_high": 3,
    "sast_moderate": 0,
    "sast_low": 0,
    "sca_high": 1,
    "sca_critical": 0,
    "sca_moderate": 0,
    "sca_low": 0
  }
}

njsscan-json: 4.809s, código 1
njsscan-sarif: 4.731s, código 1
npm-audit: 0.731s, código 1

CODIGO_DE_SAIDA=1
|
```

Versão corrigida aprovada

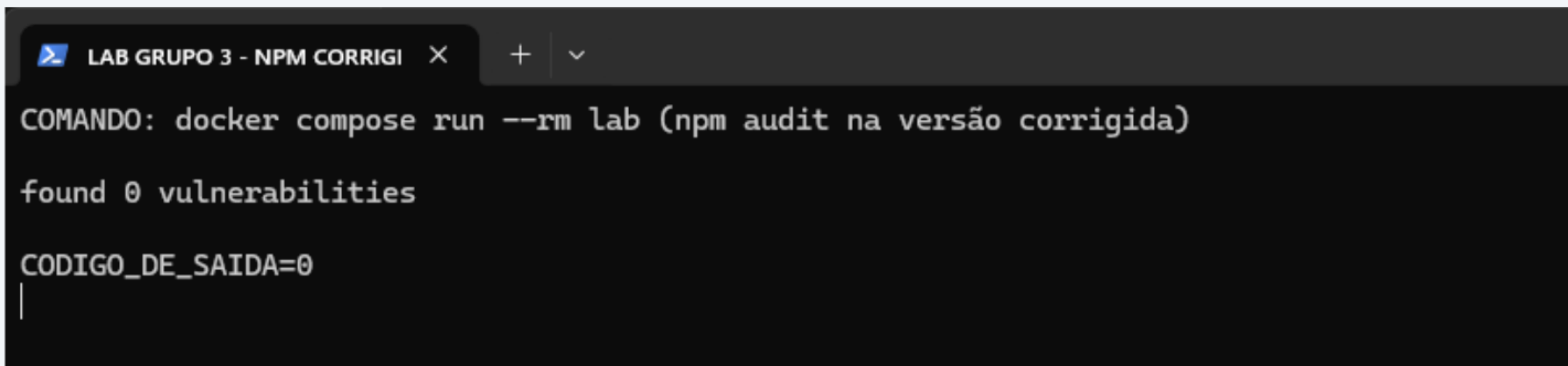
Nenhum alerta nos níveis usados pelo gate

```
LAB GRUPO 3 - TESTE APROV. X + v
COMANDO: docker compose run --rm lab corrigido

{
  "alvo": "corrigido",
  "status": "APROVADO",
  "exit_code": 0,
  "politica": "SAST ERROR = HIGH por decisão do grupo; SCA HIGH/CRITICAL bloqueiam",
  "contagens": {
    "sast_high": 0,
    "sast_moderate": 0,
    "sast_low": 0,
    "sca_high": 0,
    "sca_critical": 0,
    "sca_moderate": 0,
    "sca_low": 0
  },
  "achados_sast": []
}
njsscan-json: terminou em 5.077s, código 0
njsscan-sarif: terminou em 4.825s, código 0
npm-audit: terminou em 0.655s, código 0
|
```

Dependências depois da correção

npm audit: zero vulnerabilidades conhecidas



```
LAB GRUPO 3 - NPM CORRIGI X + v
COMANDO: docker compose run --rm lab (npm audit na versão corrigida)
found 0 vulnerabilities
CODIGO_DE_SAIDA=0
|
```

Comparação das ferramentas

- njsscan: encontra padrões perigosos no código
- npm audit: encontra dependências com avisos
- Datree: revisa políticas de Kubernetes
- StackHawk: observa a aplicação funcionando
- As ferramentas se completam

O que aprendemos

- Código e dependências exigem correções diferentes
- O scanner aponta o local; a equipe confirma o risco
- Um gate transforma a política em uma decisão clara
- A nova execução comprova a correção analisada
- Outros testes e a revisão humana continuam necessários

Demonstração e perguntas

- Executar: `docker compose run --rm lab inseguro`
- Mostrar o bloqueio e os três achados
- Executar: `docker compose run --rm lab corrigido`
- Mostrar a aprovação e explicar as correções
- Perguntas